EXP 9	A PYTHON PROGRAM TO PERFORM CRITICAL PATH METHOD ANALYSIS

AIM:

To perform Critical Path Method (CPM) Analysis for project scheduling and management using Python.

PROCEDURE:

1. **Install Required Libraries:**

o Make sure you have pandas, networkx, and matplotlib installed. You can install them using pip if not already installed.

bash

Copy code

pip install pandas networkx matplotlib

2. **Define the Problem:**

o Identify the tasks, their durations, and dependencies.

3. **Set Up the Problem in Python:**

o Use NetworkX to represent the project tasks and their dependencies. o Use the topological sort to identify the critical path.

4. **Run the Code:**

o Execute the Python code to get the critical path and project duration.

5. **Analyze the Output:**

o Interpret the results and validate the solution against the project schedule

PROBLEM 1:

Determine the Critical Path of a project using CPM.

The project has the following activities:

Task	Duration	Dependencies
A	2	—
B	3	A
C	2	A
D	4	B
E	3	C
F	2	D, E

Tasks:

1. **Construct the project network.**
2. **Find ES, EF, LS, LF for each activity.**
3. **Identify the Critical Path and total project duration.**

CODE:

```
import networkx as nx
import matplotlib.pyplot as plt
# Define tasks
tasks = {
    'A': {'duration': 2, 'dependencies': []},
    'B': {'duration': 3, 'dependencies': ['A']},
    'C': {'duration': 2, 'dependencies': ['A']},
    'D': {'duration': 4, 'dependencies': ['B']},
    'E': {'duration': 3, 'dependencies': ['C']},
    'F': {'duration': 2, 'dependencies': ['D', 'E']}
}
```

```
# Create graph
G = nx.DiGraph()
# Add nodes and edges
for task, data in tasks.items():
    G.add_node(task, duration=data['duration'])
    for dep in data['dependencies']:
        G.add_edge(dep, task)
# Topological sort
topo = list(nx.topological_sort(G))
# Earliest times
ES = {task: 0 for task in topo}
EF = {}
for task in topo:
    EF[task] = ES[task] + G.nodes[task]['duration']
    for succ in G.successors(task):
        ES[succ] = max(ES[succ], EF[task])
# Latest times
LF = {task: EF[task] for task in topo}
LS = {task: 0 for task in topo}
for task in reversed(topo):
    LS[task] = LF[task] - G.nodes[task]['duration']
    for pred in G.predecessors(task):
        LF[pred] = min(LF[pred], LS[task])
# Critical path
critical_path = [task for task in topo if ES[task] == LS[task]]
# Output
print("Earliest Start:", ES)
```

```
print("Earliest Finish:", EF)
print("Latest Start:", LS)
print("Latest Finish:", LF)
print("Critical Path:", critical_path)
# Plot
pos = nx.spring_layout(G)
nx.draw(G, pos, with_labels=True, node_size=3000)
plt.show()
```

OUTPUT:

Earliest Start: {'A': 0, 'B': 2, 'C': 2, 'D': 5, 'E': 4, 'F': 9}

Earliest Finish: {'A': 2, 'B': 5, 'C': 4, 'D': 9, 'E': 7, 'F': 11}

Latest Start: {'A': 0, 'B': 2, 'C': 4, 'D': 5, 'E': 6, 'F': 9}

Latest Finish: {'A': 2, 'B': 5, 'C': 6, 'D': 9, 'E': 9, 'F': 11}

Critical Path: ['A', 'B', 'D', 'F']

PROBLEM 2: CPM METHOD

A project consists of the following activities with durations:

Activity	Duration (days)	Predecessors
A	3	—
B	2	A
C	4	A
D	6	B
E	5	B, C
F	3	D, E

Activity	Duration (days)	Predecessors
G	2	F

CODE:

```
import networkx as nx
import matplotlib.pyplot as plt
class Task:
    def __init__(self, name, duration):
        self.name = name
        self.duration = duration
        self.ES = 0
        self.EF = 0
        self.LS = float('inf')
        self.LF = float('inf')
        self.successors = []

class CPM:
    def __init__(self, tasks, dependencies):
        self.tasks = {k: Task(k, v) for k, v in tasks.items()}
        self.dependencies = dependencies
        self.build_graph()
    def build_graph(self):
        for task, deps in self.dependencies.items():
            for dep in deps:
                self.tasks[dep].successors.append(self.tasks[task])
    def forward_pass(self):
        for task in self.tasks.values():
```

```
        if not self.dependencies[task.name]:
            task.ES = 0
            task.EF = task.duration
    for task in self.tasks.values():
        for succ in task.successors:
            if succ.ES < task.EF:
                succ.ES = task.EF
                succ.EF = succ.ES + succ.duration
def backward_pass(self):
    max_time = max(t.EF for t in self.tasks.values())

    for task in self.tasks.values():
        if not task.successors:
            task.LF = max_time
            task.LS = task.LF - task.duration
    for task in reversed(list(self.tasks.values())):
        for succ in task.successors:
            if task.LF > succ.LS:
                task.LF = succ.LS
                task.LS = task.LF - task.duration
def critical_path(self):
    return [t.name for t in self.tasks.values() if t.ES == t.LS]
def visualize(self):
    G = nx.DiGraph()
    for t in self.tasks.values():
        label = f"{t.name}\nES:{t.ES} EF:{t.EF}\nLS:{t.LS} LF:{t.LF}"
        G.add_node(t.name, label=label)
```

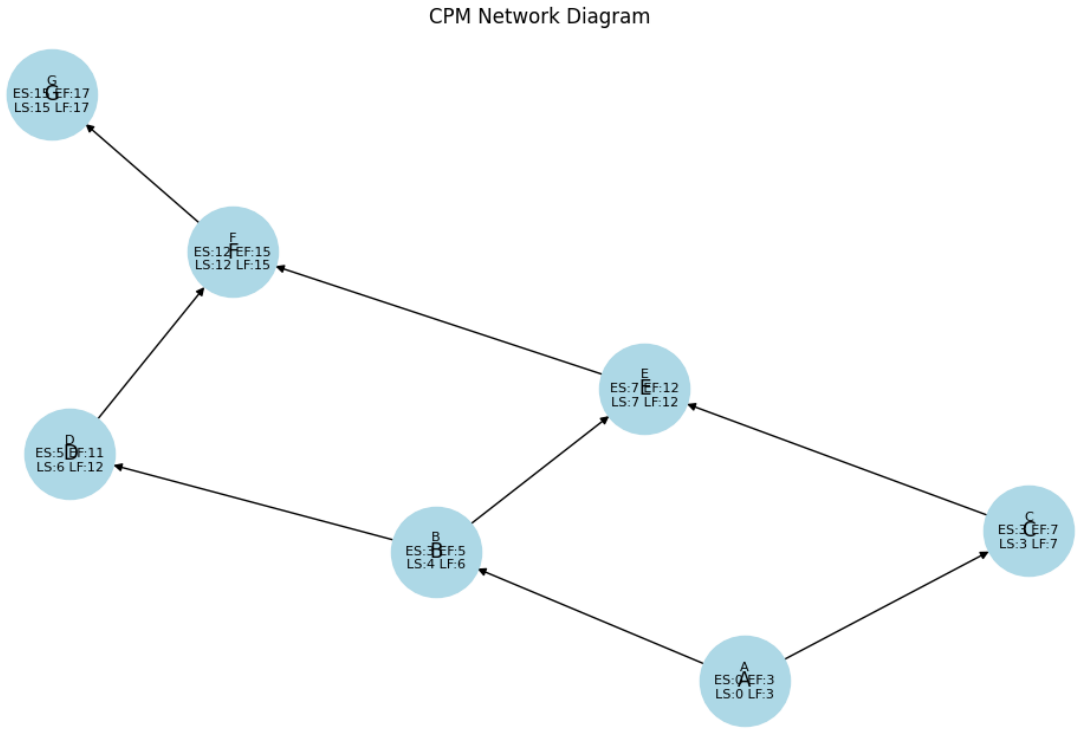
```
        for t in self.tasks.values():
            for s in t.successors:
                G.add_edge(t.name, s.name)
        pos = nx.spring_layout(G)
        labels = nx.get_node_attributes(G, 'label')
        plt.figure(figsize=(10,6))
        nx.draw(G, pos, with_labels=True,
                node_size=3000,
                node_color="lightblue")
        nx.draw_networkx_labels(G, pos, labels, font_size=8)
        plt.title("CPM Network Diagram")
        plt.show()
# INPUT
tasks = {
    'A': 3, 'B': 2, 'C': 4,
    'D': 6, 'E': 5, 'F': 3, 'G': 2
}
dependencies = {
    'A': [],
    'B': ['A'],
    'C': ['A'],
    'D': ['B'],
    'E': ['B', 'C'],
    'F': ['D', 'E'],
    'G': ['F']
}
# RUN
```

```
cpm = CPM(tasks, dependencies)
cpm.forward_pass()
cpm.backward_pass()
cp = cpm.critical_path()
print("Critical Path:", " -> ".join(cp))
print("Project Duration:", max(t.EF for t in cpm.tasks.values()))
cpm.visualize()
```

OUTPUT:

Critical Path: A -> C -> E -> F -> G

Project Duration: 17



PROBLEM 3: SHORT CPM PROBLEM

Activity	Duration	Predecessor
A	2	—

Activity	Duration	Predecessor
B	3	A
C	4	A
D	2	B, C

CODE:

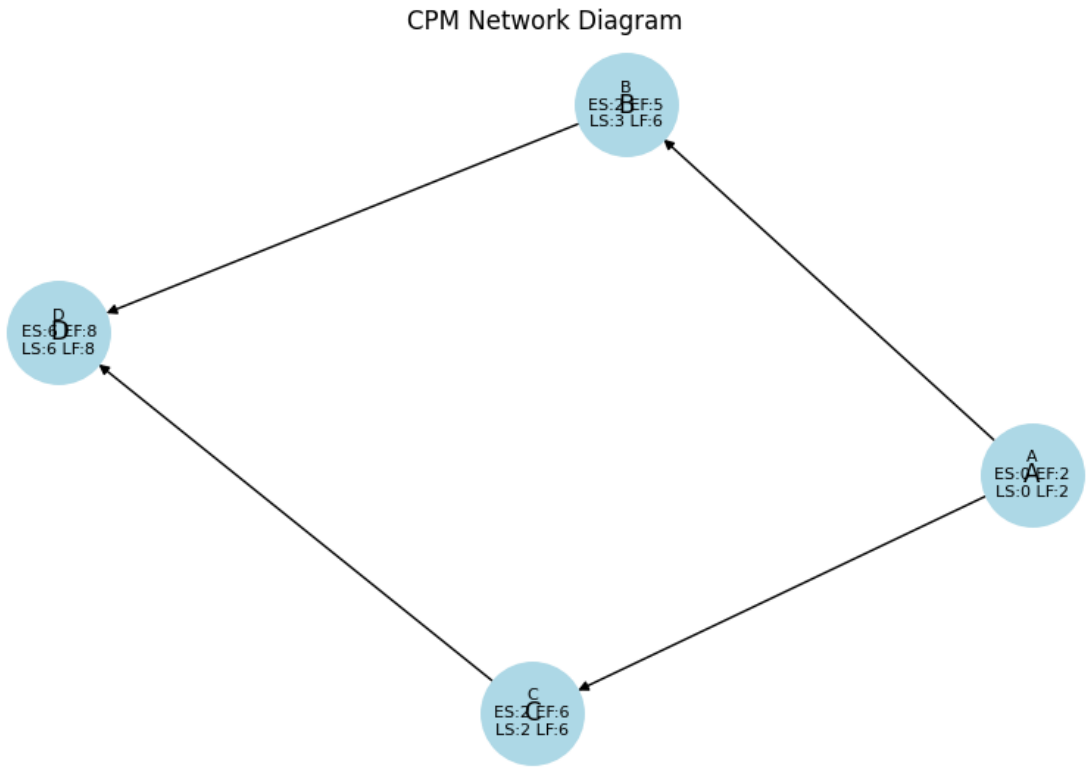
```
import networkx as nx
tasks = {'A':2, 'B':3, 'C':4, 'D':2}
deps = {'A':[], 'B':['A'], 'C':['A'], 'D':['B','C']}
ES, EF = {}, {}
# Forward pass
for t in tasks:
    ES[t] = max([EF[p] for p in deps[t]] or [0])
    EF[t] = ES[t] + tasks[t]
# Backward pass
LF, LS = {}, {}
project_time = max(EF.values())
for t in reversed(list(tasks)):
    if t not in sum(deps.values(), []):
        LF[t] = project_time
    else:
        LF[t] = min([LS[s] for s in tasks if t in deps[s]])
    LS[t] = LF[t] - tasks[t]



# Critical Path
cp = [t for t in tasks if ES[t] == LS[t]]
print("ES:", ES)
```

```
print("EF:", EF)
print("LS:", LS)
print("LF:", LF)
print("Critical Path:", " -> ".join(cp))
print("Project Duration:", project_time)
```

OUTPUT:



Critical Path: A -> C -> D

Project Duration: 8

RESULT:

Thus Critical Path Method (CPM) Analysis For Project Scheduling And Management Using Python Has Been Successfully Implemented.